

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

**AVINASH ACHARYA
(1BM24CS063)**

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **AVINASH ACHARYA (1BM24CS063)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Syed Akram
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-5
2	Implement Johnson Trotter algorithm to generate permutations.	6-8
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	9-11
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	12-14
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	15-17
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	18-19
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	20-21
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	22-26
9	Implement Fractional Knapsack using Greedy technique.	27-29
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	30-31
11	Implement "N-Queens Problem" using Backtracking.	32-33
12	Leetcode Problems	34-36

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include<stdio.h>
#define MAX 100
int main()
{
    int n, adj[MAX][MAX], indegree[MAX]={0};
    int queue[MAX], front=0, rear=0;
    int topo[MAX], count=0;

    printf("Enter the number of vertices: ");
    scanf("%d",&n);

    printf("Enter the adjacency matrix:\n");
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
        {
            scanf("%d",&adj[i][j]);
            if(adj[i][j]==1)
                indegree[j]++;
        }

    for(int i=0;i<n;i++)
        if(indegree[i]==0)
            queue[rear++]=i;

    while(front<rear)
    {
        int u=queue[front++];
        topo[count++]=u;

        for(int v=0;v<n;v++)
            if(adj[u][v]==1)
            {
                indegree[v]--;
                if(indegree[v]==0)
                    queue[rear++]=v;
            }
    }

    if(count!=n)
        printf("Topological sorting is not possible.\n");
    else
    {
        printf("Topological ordering is: ");
        for(int i=0;i<n;i++)
            printf("%d ",topo[i]);
    }

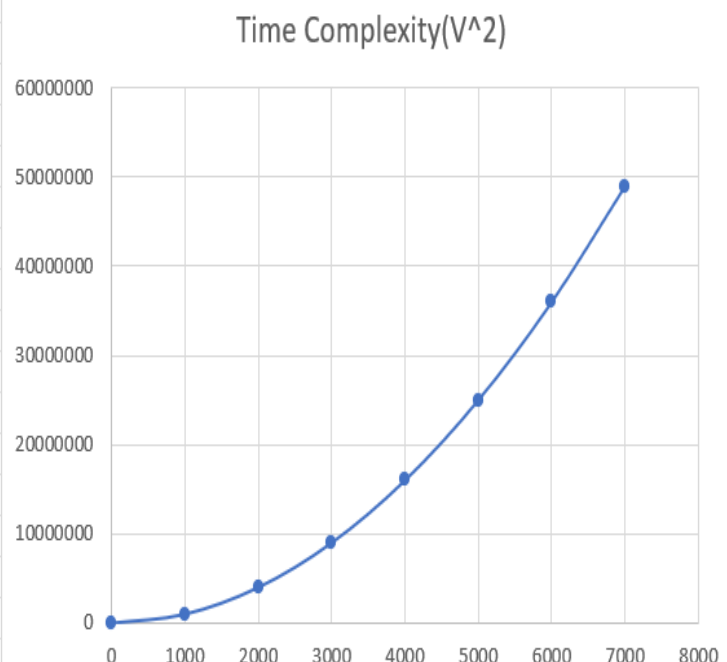
    return 0;
}
```

```

Enter the number of vertices: 4
Enter the adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0
Topological ordering is: 0 1 2 3

Process returned 0 (0x0)   execution time : 20.388 s
Press any key to continue.

```

[illegible]

LAB PROGRAM 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1

int getMobileIndex(int a[], int dir[], int n)
{
    int mobile = 0;
    int mobileIndex = -1;
    for (int i = 0; i < n; i++)
    {
        if (dir[a[i]] == LEFT && i != 0)
        {
            if (a[i] > a[i - 1] && a[i] > mobile)
            {
                mobile = a[i];
                mobileIndex = i;
            }
        }

        if (dir[a[i]] == RIGHT && i != n - 1)
        {
            if (a[i] > a[i + 1] && a[i] > mobile)
            {
                mobile = a[i];
                mobileIndex = i;
            }
        }
    }

    return mobileIndex;
}

void print(int a[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
}

void johnsonTrotter(int n)
{
    int a[n];
    int dir[n + 1];

    for (int i = 0; i < n; i++)
        a[i] = i + 1;
```

```

for (int i = 1; i <= n; i++)
    dir[i] = LEFT;

print(a, n);

while (1)
{
    int mobileIndex = getMobileIndex(a, dir, n);

    if (mobileIndex == -1)
        break;

    int mobile = a[mobileIndex];

    if (dir[mobile] == LEFT)
    {
        int temp = a[mobileIndex];
        a[mobileIndex] = a[mobileIndex - 1];
        a[mobileIndex - 1] = temp;

        mobileIndex--;
    }
    else
    {
        int temp = a[mobileIndex];
        a[mobileIndex] = a[mobileIndex + 1];
        a[mobileIndex + 1] = temp;

        mobileIndex++;
    }

    for (int i = 0; i < n; i++)
    {
        if (a[i] > mobile)
            dir[a[i]] *= -1;
    }

    print(a, n);
}

int main()
{
    int n;

    printf("Enter the value of n: ");
    scanf("%d", &n);

    johnsonTrotter(n);
    return 0;
}

```

OUTPUT:

```
"C:\Users\avina\OneDrive\De:  ×  +  v
Enter the value of n: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Process returned 0 (0x0)   execution time : 4.106 s
Press any key to continue.
```

LAB PROGRAM 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int arr[], int left, int mid, int right)
{
    int temp[50];
    int i = left;
    int j = mid + 1;
    int k = left;

    while (i <= mid && j <= right)
    {
        if (arr[i] < arr[j])
        {
            temp[k++] = arr[i++];
        }
        else
        {
            temp[k++] = arr[j++];
        }
    }

    while (i <= mid)
    {
        temp[k++] = arr[i++];
    }

    while (j <= right)
    {
        temp[k++] = arr[j++];
    }

    for (i = left; i <= right; i++)
    {
        arr[i] = temp[i];
    }
}

void mergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}
```

```

}

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements:\n");

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    clock_t start, end;
    double cpu_time_used;

    start = clock();

    mergeSort(arr, 0, n - 1);

    end = clock();

    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\nSorted array:\n");

    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n\nExecution Time = %f seconds\n", cpu_time_used);

    return 0;
}

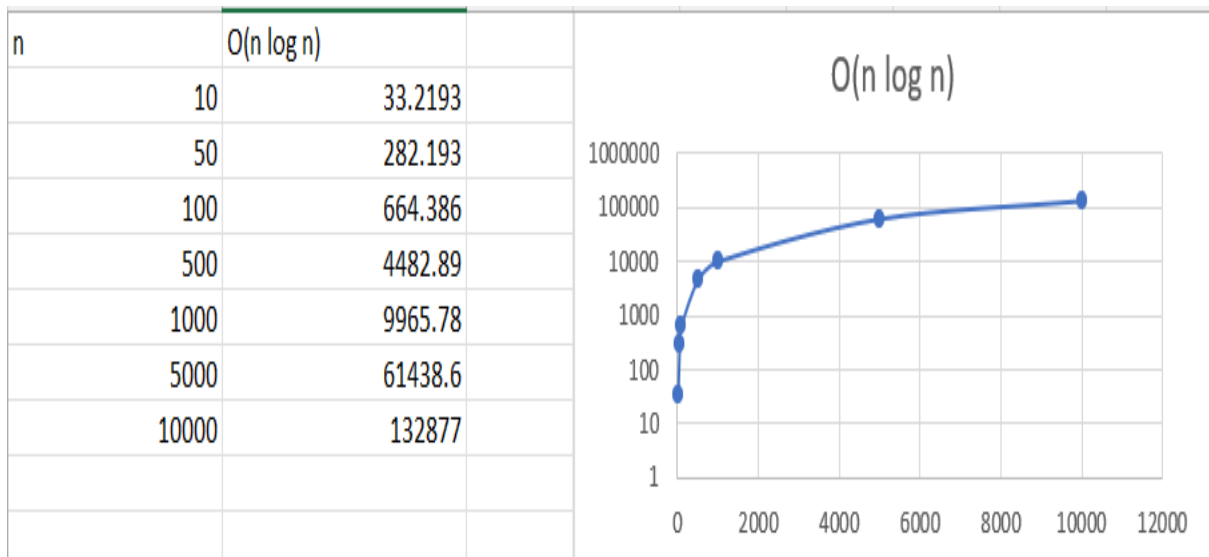
```

OUTPUT:

```
"C:\Users\avina\OneDrive\De: X + v
Enter number of elements: 7
Enter the elements:
12 34 96 7 45 33 88

Sorted array:
7 12 33 34 45 88 96

Time Taken = 0.000000 ms
```



LAB PROGRAM 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken

```
#include <stdio.h>
#include <time.h>

void swap(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quick_sort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);

        quick_sort(arr, low, p - 1);
        quick_sort(arr, p + 1, high);
    }
}

int main()
{
    int n;

    printf("Enter size of array: ");
    scanf("%d", &n);

    int arr[n];
```

```
printf("Enter array elements:\n");

for (int i = 0; i < n; i++)
{
    scanf("%d", &arr[i]);
}

clock_t start, end;

start = clock();

quick_sort(arr, 0, n - 1);

end = clock();

double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;

printf("\nSorted array:\n");

for (int i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}

printf("\n\nTime Taken = %.6f ms\n", time_taken);

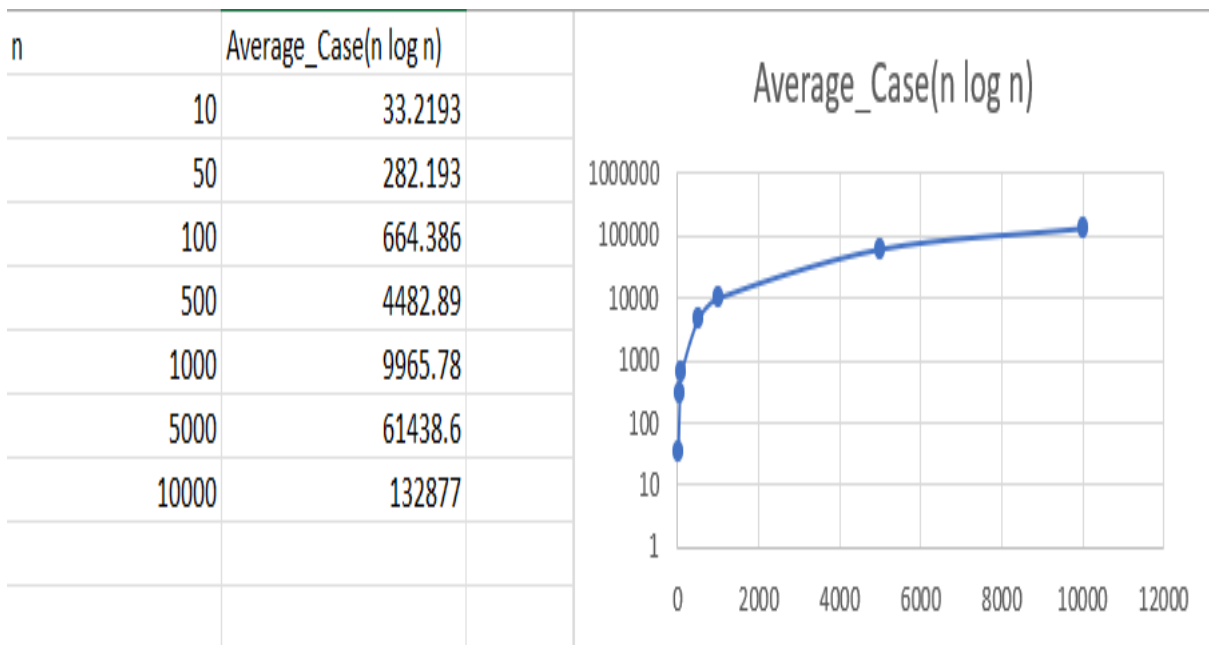
return 0;
}
```

OUTPUT:

```
"C:\Users\avina\OneDrive\De: X + v
Enter size of array: 5
Enter array elements:
-14 12 3 0 45

Sorted array:
-14 0 3 12 45

Time Taken = 0.000000 ms
```



LAB PROGRAM 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>
#include <time.h>

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    int i, temp;

    for (i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (i = n - 1; i > 0; i--)
    {
        temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

void main()
{
```

```
int n, i;
clock_t start, end;
double time_taken;

printf("Enter number of elements: ");
scanf("%d", &n);

int arr[n];

printf("\nEnter elements:\n");

for (i = 0; i < n; i++)
    scanf("%d", &arr[i]);

start = clock();

heapSort(arr, n);

end = clock();

printf("\nSorted array:\n");

for (i = 0; i < n; i++)
    printf("%d ", arr[i]);

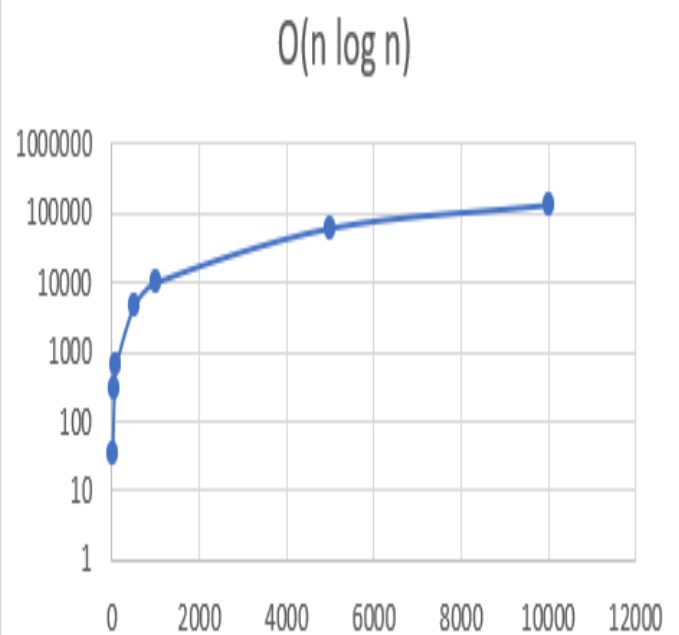
time_taken = (((double)(end - start)) / CLOCKS_PER_SEC) * 1000;

printf("\n\nTime taken = %f ms\n", time_taken);
}
```

OUTPUT:

```
"C:\Users\avina\OneDrive\De:  ×  +  v
Enter number of elements: 5
Enter elements:
-15 34 0 -12 16
Sorted array:
-15 -12 0 16 34
Time taken = 0.000000 ms
```

n	$O(n \log n)$	
10	33.2193	
50	282.193	
100	664.386	
500	4482.89	
1000	9965.78	
5000	61438.6	
10000	132877	



LAB PROGRAM 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>
#include <time.h>

int max(int a, int b)
{
    return (a > b) ? a : b;
}

void knapsack(int n, int W, int wt[], int val[], int table[n + 1][W + 1])
{
    for (int i = 0; i <= n; i++)
        table[i][0] = 0;

    for (int j = 0; j <= W; j++)
        table[0][j] = 0;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= W; j++)
        {
            if (j < wt[i - 1])
            {
                table[i][j] = table[i - 1][j];
            }
            else
            {
                table[i][j] = max(table[i - 1][j],
                                   val[i - 1] + table[i - 1][j - wt[i - 1]]);
            }
        }
    }
}

void printResult(int n, int W, int table[n + 1][W + 1])
{
    printf("\nDP Table:\n\n");

    for (int i = 0; i <= n; i++)
    {
        for (int j = 0; j <= W; j++)
        {
            printf("%d ", table[i][j]);
        }
        printf("\n");
    }
    printf("\nMaximum profit = %d\n", table[n][W]);
}

int main()
{
    int n, W;
    printf("Enter number of items: ");
```

```

scanf("%d", &n);

int wt[n], val[n];

printf("Enter the weights:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &wt[i]);

printf("Enter the values:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &val[i]);

printf("Enter knapsack capacity: ");
scanf("%d", &W);

int table[n + 1][W + 1];

clock_t start, end;

start = clock();

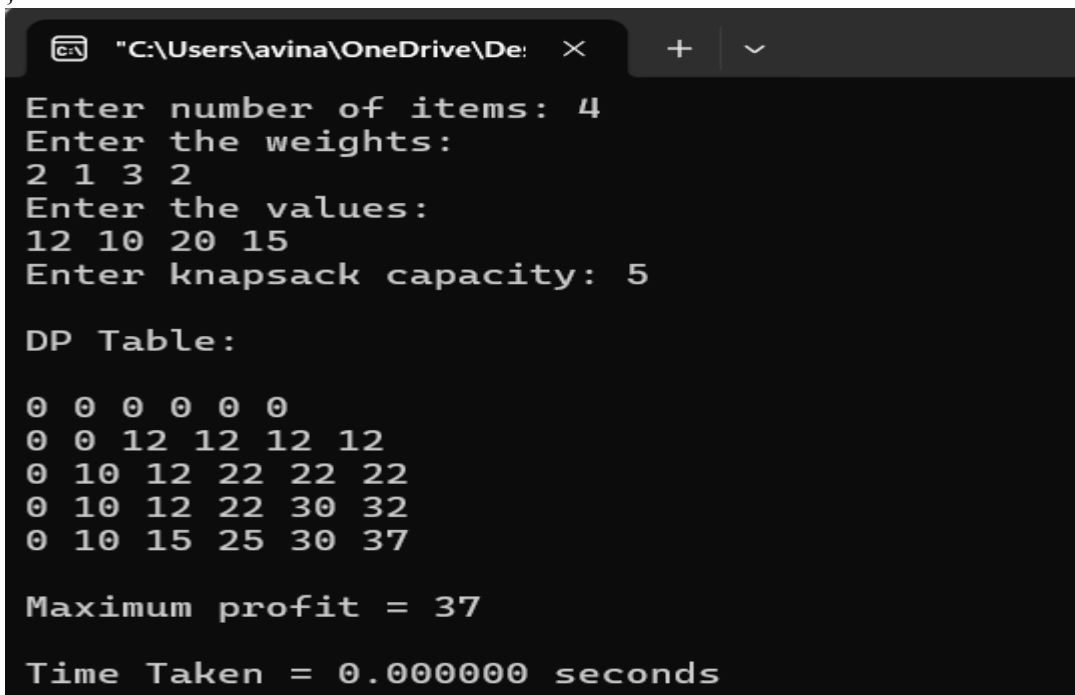
knapsack(n, W, wt, val, table);

end = clock();

printResult(n, W, table);

double time_taken = ((double)(end - start) / CLOCKS_PER_SEC);
printf("\nTime Taken = %.6f seconds\n", time_taken);
return 0;
}

```



```

C:\Users\avina\OneDrive\De: x + v
Enter number of items: 4
Enter the weights:
2 1 3 2
Enter the values:
12 10 20 15
Enter knapsack capacity: 5

DP Table:

0 0 0 0 0 0
0 0 12 12 12 12
0 10 12 22 22 22
0 10 12 22 30 32
0 10 15 25 30 37

Maximum profit = 37

Time Taken = 0.000000 seconds

```

LAB PROGRAM 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>
#include <time.h>

#define INF 999

void main()
{
    int n, i, j, k;
    clock_t start, end;
    double time_taken;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int dist[n][n];

    printf("\nEnter adjacency matrix: ");
    printf("(Use %d for INF)\n", INF);

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &dist[i][j]);
        }
    }

    start = clock();

    for (k = 0; k < n; k++)
    {
        for (i = 0; i < n; i++)
        {
            for (j = 0; j < n; j++)
            {
                if (dist[i][k] + dist[k][j] < dist[i][j])
                {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    end = clock();

    printf("\nShortest Distance Matrix:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (dist[i][j] == INF)
```

```

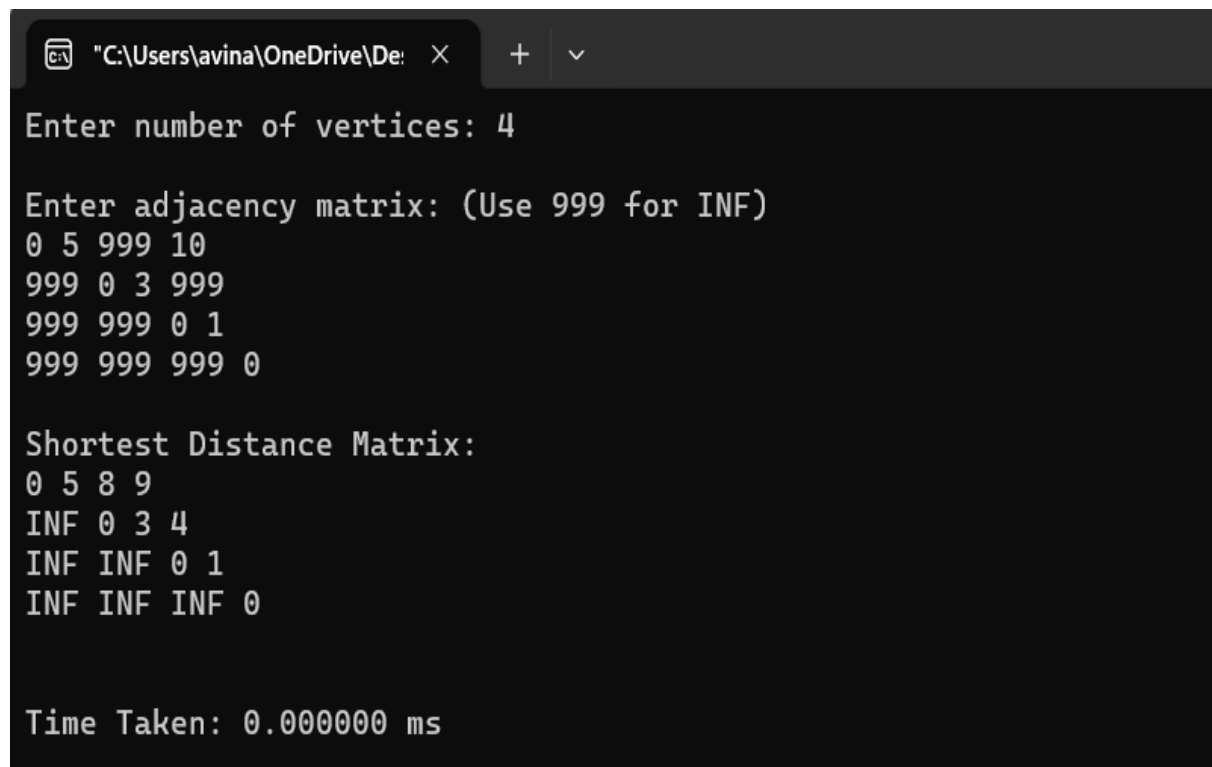
        printf("INF ");
    else
        printf("%d ", dist[i][j]);
    }
    printf("\n");
}

time_taken = (((double)(end - start)) / CLOCKS_PER_SEC) * 1000;

printf("\n\nTime Taken: %f ms\n", time_taken);
}

```

OUTPUT:



```

C:\Users\avina\OneDrive\De:
Enter number of vertices: 4

Enter adjacency matrix: (Use 999 for INF)
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0

Shortest Distance Matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

Time Taken: 0.000000 ms

```

LAB PROGRAM 8: (PART a)

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>
#include <time.h>

int main()
{
    int n, i, j;
    int graph[10][10];
    int visited[10] = {0};
    int edgeCount = 0;
    int totalCost = 0;

    clock_t start, end;
    double time_taken;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);

            if (graph[i][j] == 0 && i != j)
            {
                graph[i][j] = 999;
            }
        }
    }

    start = clock();

    visited[0] = 1;

    printf("\nEdges selected for MST:\n");

    while (edgeCount < n - 1)
    {
        int min = 999;
        int x = -1, y = -1;

        for (i = 0; i < n; i++)
        {
            if (visited[i])
            {
                for (j = 0; j < n; j++)
                {
                    if (!visited[j] && graph[i][j] < min)
                    {
                        min = graph[i][j];
                        x = i;
                        y = j;
                    }
                }
            }
        }
    }
}
```

```

    }
}

if (x != -1 && y != -1)
{
    printf("%d --> %d cost = %d\n", x, y, min);

    totalCost += min;
    visited[y] = 1;
    edgeCount++;
}
}

end = clock();

printf("\nMinimum Cost = %d\n", totalCost);

time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;

printf("Time Taken = %f ms\n", time_taken);

return 0;
}

```

OUTPUT:

```

C:\Users\avina\OneDrive\De:
Enter number of vertices: 6
Enter adjacency matrix:
0 4 6 0 0 0
4 0 6 3 4 0
6 6 0 1 8 0
0 3 1 0 2 3
0 4 8 2 0 7
0 0 0 3 7 0

Edges selected for MST:
0 --> 1 cost = 4
1 --> 3 cost = 3
3 --> 2 cost = 1
3 --> 4 cost = 2
3 --> 5 cost = 3

Minimum Cost = 13
Time Taken = 1.000000 ms

```

LAB PROGRAM 8: (PART b)

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>
#include <time.h>

int set[10];

int getParent(int node)
{
    while (set[node] != 0)
    {
        node = set[node];
    }
    return node;
}

int connect(int x, int y)
{
    if (x != y)
    {
        set[y] = x;
        return 1;
    }
    return 0;
}

int main()
{
    int graph[10][10];
    int n;
    int i, j;
    int minCost = 0;
    int edgeCount = 0;
    int a, b;
    int p, q;
    clock_t start, end;
    double time_taken;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }
}
```

```

        if (graph[i][j] == 0)
        {
            graph[i][j] = 999;
        }
    }
}

start = clock();

printf("\nEdges in Minimum Spanning Tree:\n");

while (edgeCount < n - 1)
{
    int min = 999;

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (graph[i][j] < min)
            {
                min = graph[i][j];
                a = p = i;
                b = q = j;
            }
        }
    }

    p = getParent(p);
    q = getParent(q);

    if (connect(p, q))
    {
        printf("%d --> %d cost = %d\n", a, b, min);

        minCost += min;
        edgeCount++;
    }

    graph[a][b] = graph[b][a] = 999;
}

end = clock();

printf("\nMinimum Cost = %d\n", minCost);

time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;

printf("Time Taken = %f ms\n", time_taken);

return 0;
}

```

OUTPUT:

```
"C:\Users\avina\OneDrive\De" X + v
Enter number of vertices: 5
Enter adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edges in Minimum Spanning Tree:
0 --> 1 cost = 2
1 --> 2 cost = 3
1 --> 4 cost = 5
0 --> 3 cost = 6

Minimum Cost = 16
Time Taken = 1.000000 ms
```

LAB PROGRAM 9:

Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>
#include <time.h>

float knapsack_greedy(int p[], int w[], int m, int n)
{
    float x[n], max_profit = 0.0;

    for (int i = 0; i < n; i++)
    {
        x[i] = 0.0;
    }

    int rem_cap = m, i = 0;

    while (rem_cap > 0)
    {
        if (w[i] > rem_cap)
            break;

        x[i] = 1.0;
        rem_cap = rem_cap - w[i];
        i++;
    }

    if (rem_cap != 0 && i < n)
    {
        x[i] = (float)rem_cap / w[i];
    }

    for (int i = 0; i < n; i++)
    {
        max_profit = max_profit + (x[i] * p[i]);
    }

    return max_profit;
}

void main()
{
    int n, m;
    clock_t start, end;
    double time_taken;

    printf("Enter number of objects: ");
    scanf("%d", &n);

    printf("Enter max capacity of Knapsack: ");
    scanf("%d", &m);
```

```

int p[n], w[n];
float wp[n], profit;

printf("\nEnter profit, weight for %d objects\n", n);

for (int i = 0; i < n; i++)
{
    scanf("%d %d", &p[i], &w[i]);
    wp[i] = (float)p[i] / w[i];
}

for (int i = 0; i < n; i++)
{
    for (int j = i + 1; j < n; j++)
    {
        if (wp[i] < wp[j])
        {
            float temp = wp[i];
            wp[i] = wp[j];
            wp[j] = temp;

            int temp2 = w[i];
            w[i] = w[j];
            w[j] = temp2;

            int temp3 = p[i];
            p[i] = p[j];
            p[j] = temp3;
        }
    }
}

start = clock();

profit = knapsack_greedy(p, w, m, n);

end = clock();

printf("\nMaximum profit: %f", profit);

time_taken = (((double)(end - start)) / CLOCKS_PER_SEC) * 1000;

printf("\n\nTime Taken: %f ms\n", time_taken);
}

```

OUTPUT:

```
"C:\Users\avina\OneDrive\De: X + v
Enter number of objects: 3
Enter max capacity of Knapsack: 4

Enter profit, weight for 3 objects
11 2
14 3
20 5

Maximum profit: 20.333334

Time Taken: 0.000000 ms
```

LAB PROGRAM 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>

void main()
{
    int c[100][100], s[100], dist[100], v, n, u;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &c[i][j]);

            if (c[i][j] == 0 && i != j)
                c[i][j] = 999;
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &v);

    for (int i = 0; i < n; i++)
    {
        s[i] = 0;
        dist[i] = c[v][i];
    }

    s[v] = 1;
    dist[v] = 0;

    for (int w = 0; w < n - 1; w++)
    {
        int min = 999;
        u = -1;

        for (int i = 0; i < n; i++)
        {
            if (s[i] == 0 && dist[i] < min)
            {
                min = dist[i];
                u = i;
            }
        }
    }
}
```

```

    if (u == -1)
        break;

    s[u] = 1;

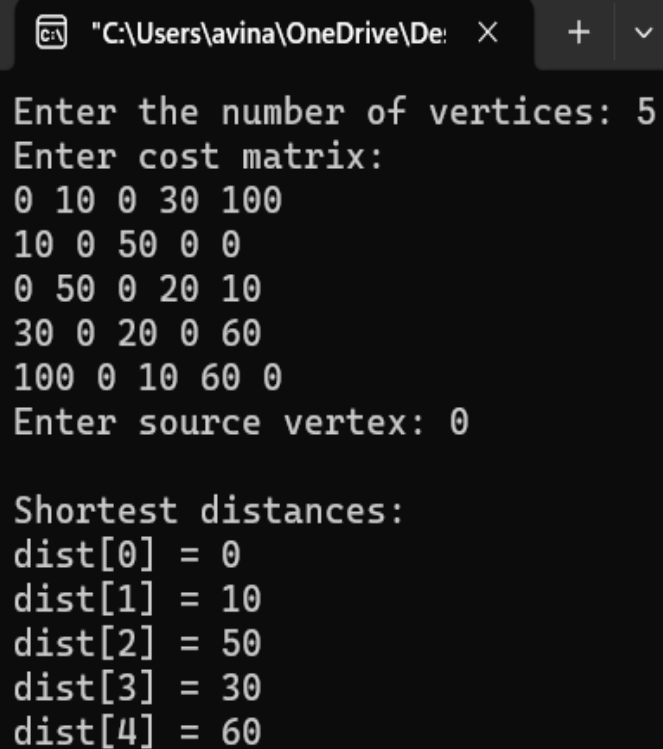
    for (int k = 0; k < n; k++)
    {
        if (s[k] == 0 && dist[k] > dist[u] + c[u][k])
        {
            dist[k] = dist[u] + c[u][k];
        }
    }
}

printf("\nShortest distances:\n");

for (int i = 0; i < n; i++)
{
    printf("dist[%d] = %d\n", i, dist[i]);
}
}

```

OUTPUT:



```

Enter the number of vertices: 5
Enter cost matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter source vertex: 0

Shortest distances:
dist[0] = 0
dist[1] = 10
dist[2] = 50
dist[3] = 30
dist[4] = 60

```

LAB PROGRAM 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int x[20], n;
int solutionCount = 0;

int placeQueen(int k, int i)
{
    for (int j = 1; j <= k - 1; j++)
    {
        if ((x[j] == i) || (abs(j - k) == abs(x[j] - i)))
            return 0;
    }
    return 1;
}

void printSolution()
{
    solutionCount++;

    printf("\nSolution %d:\n", solutionCount);

    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            if (x[i] == j)
                printf(" Q ");
            else
                printf(" . ");
        }
        printf("\n");
    }
}

void nQueens(int k)
{
    for (int i = 1; i <= n; i++)
    {
        if (placeQueen(k, i))
        {
            x[k] = i;

            if (k == n)
                printSolution();
            else
                nQueens(k + 1);
        }
    }
}
```

```

    }
}

void main()
{
    clock_t start, end;

    printf("Enter number of queens: ");
    scanf("%d", &n);

    start = clock();

    nQueens(1);

    end = clock();

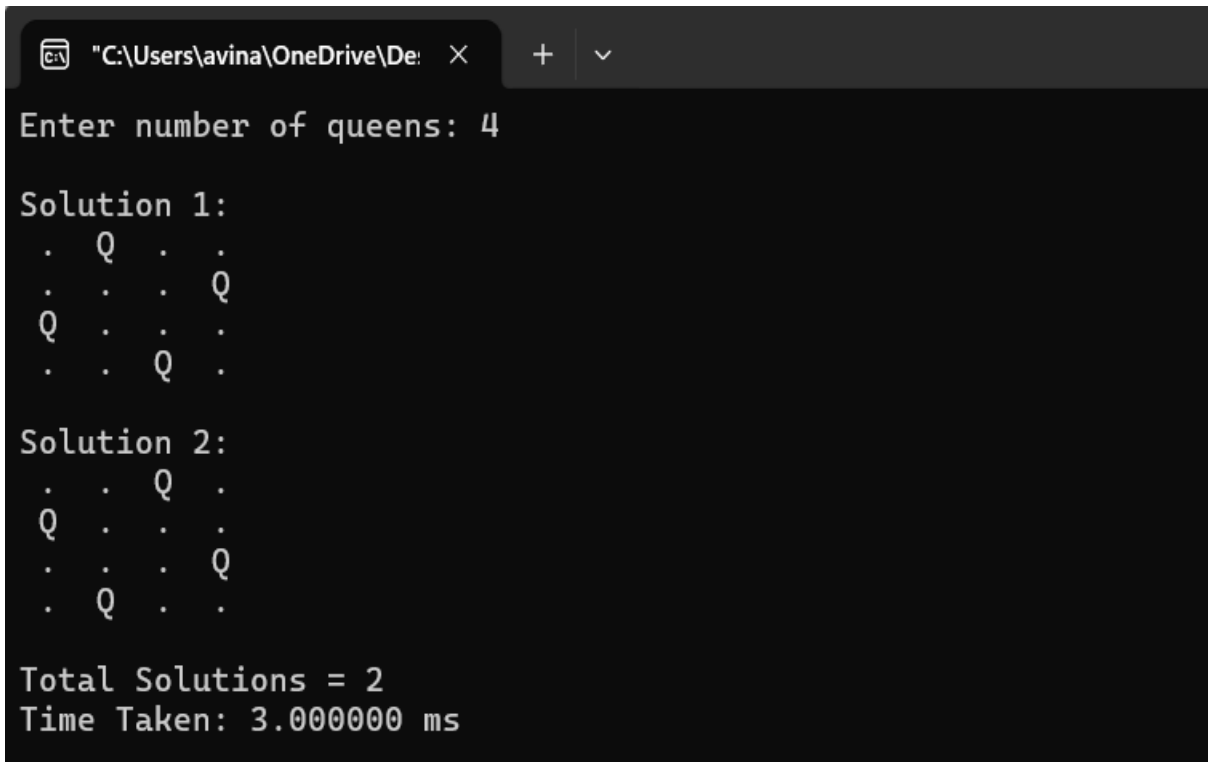
    if (solutionCount == 0)
        printf("\nNo Solution Exists\n");
    else
        printf("\nTotal Solutions = %d\n", solutionCount);

    double time_taken =
        (((double)(end - start)) / CLOCKS_PER_SEC) * 1000;

    printf("Time Taken: %f ms\n", time_taken);
}

```

OUTPUT:



```

C:\Users\avina\OneDrive\De:
Enter number of queens: 4

Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .

Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .

Total Solutions = 2
Time Taken: 3.000000 ms

```

LEETCODE PROGRAMS:

11. Container With Most Water

The screenshot shows the LeetCode interface for the 'Container With Most Water' problem. On the left, the C code is displayed, implementing a two-pointer approach. On the right, the submission status is 'Accepted' with 65/65 test cases passed. The runtime is 0 ms (Beats 100.00%) and memory is 13.99 MB (Beats 100.00%). A bar chart shows the runtime distribution, with the user's solution being the fastest.

```
1 int min(int a,int b)
2 {
3     if(a<b) return a;
4     else return b;
5 }
6 int maxArea(int* height, int heightSize) {
7     int maxArea=INT_MIN;
8     int left=0;
9     int right=heightSize-1;
10    while(left<right)
11    {
12        int area=(right-left)*min(height[left],height[right]);
13        if(area>maxArea)
14        {
15            maxArea=area;
16        }
17        if(height[left]<height[right])
18        {
19            left++;
20        }
21        else
22        {
23            right--;
24        }
25    }
26    return maxArea;
27 }
```

Accepted 65 / 65 testcases passed
Avinash_Acharya1 submitted at Feb 23, 2026 14:50

Runtime: 0 ms | Beats 100.00%
Memory: 13.99 MB | Beats 100.00%

Code | C

```
1 int min(int a,int b)
```

34. Find First and Last Position of Element in Sorted Array

The screenshot shows the LeetCode interface for the 'Find First and Last Position of Element in Sorted Array' problem. On the left, the C code is displayed, implementing a linear search. On the right, the submission status is 'Accepted' with 88/88 test cases passed. The runtime is 0 ms (Beats 100.00%) and memory is 9.88 MB (Beats 100.00%). A bar chart shows the runtime distribution, with the user's solution being the fastest.

```
1 |
2 int* searchRange(int* nums, int numsSize, int target, int*
3     returnSize) {
4     int *ans=(int*)malloc(2*sizeof(int));
5     *returnSize=2;
6     ans[0]=-1;
7     ans[1]=-1;
8     for(int i=0;i<numsSize;i++)
9     {
10        if(nums[i]==target)
11        {
12            ans[0]=i;
13            while(i+1<numsSize && nums[i+1]==target)
14            {
15                i++;
16            }
17            ans[1]=i;
18            break;
19        }
20    }
21    return ans;
22 }
```

Accepted 88 / 88 testcases passed
Avinash_Acharya1 submitted at Feb 23, 2026 14:26

Runtime: 0 ms | Beats 100.00%
Memory: 9.88 MB | Beats 100.00%

Code | C

268. Missing Number

Code

C v Auto

1

2

3

4

5

6

7

8

9

10

11

12

13

```
int missingNumber(int* nums, int numsSize) {
    int XOR_all=0;
    int XOR=0;
    for(int i=0;i<numsSize;i++)
    {
        XOR^=nums[i];
    }
    for(int i=0;i<numsSize+1;i++)
    {
        XOR_all^=i;
    }
    return XOR_all^XOR;
}
```

Accepted x

Accepted 122 / 122 testcases passed

Avinash_Acharya1 submitted at Mar 02, 2026 14:12

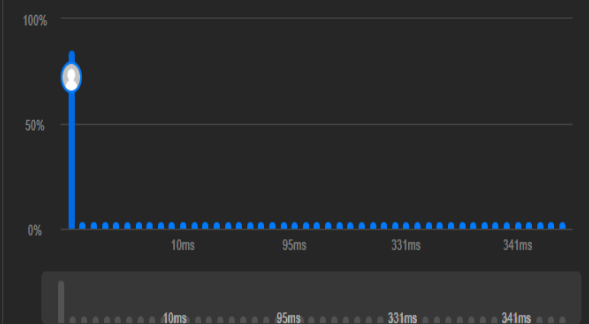
Analysis Solution

Runtime

0 ms | Beats 100.00%

Memory

9.66 MB | Beats 43.54%



207. Course Schedule

Code

C v Auto

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int prerequisitesColSize) {
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* indegree = (int*)calloc(numCourses, sizeof(int));
    int* outCount = (int*)calloc(numCourses, sizeof(int));

    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(numCourses * sizeof(int));
    }

    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        int prereq = prerequisites[i][1];

        graph[prereq][outCount[prereq]++] = course;
        indegree[course]++;
    }

    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;

    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0)
            queue[rear++] = i;
    }

    int count = 0;
}
```

Accepted x

Accepted 54 / 54 testcases passed

Avinash_Acharya1 submitted at Mar 02, 2026 15:11

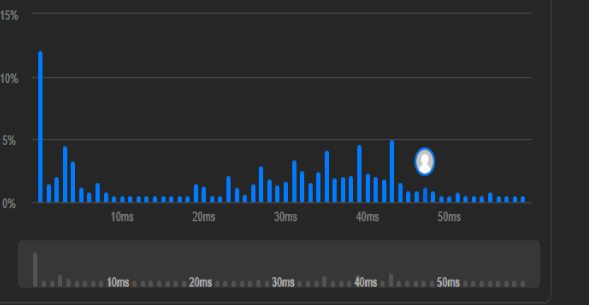
Analysis Solution

Runtime

47 ms | Beats 12.04%

Memory

110.00 MB | Beats 25.38%



Code | C

```
1 #define M 2000
2 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, i
3     int** graph = (int**)malloc(numCourses * sizeof(int*));
4     int* indegree = (int*)calloc(numCourses, sizeof(int));
5     int* outCount = (int*)calloc(numCourses, sizeof(int));
6
```

801. Minimum Swaps To Make Sequences Increasing

Code

C v Auto

```
1 int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
2     int keep = 0;
3     int swap = 1;
4
5     for(int i = 1; i < nums1Size; i++)
6     {
7         int newKeep = 100000;
8         int newSwap = 100000;
9
10        if(nums1[i] > nums1[i-1] && nums2[i] > nums2[i-1])
11        {
12            if(newKeep > keep)
13                newKeep = keep;
14
15            if(newSwap > swap + 1)
16                newSwap = swap + 1;
17        }
18
19        if(nums1[i] > nums2[i-1] && nums2[i] > nums1[i-1])
20        {
21            if(newKeep > swap)
22                newKeep = swap;
23
24            if(newSwap > keep + 1)
25                newSwap = keep + 1;
26        }
27
28        keep = newKeep;
29        swap = newSwap;
30    }
31 }
```

Accepted X

Accepted 117 / 117 testcases passed

Avinash_Acharya1 submitted at Apr 27, 2026 15:41

Analysis Solution

✓ Approach ✓ Efficiency ✓ Code Style

Congratulations! You passed on your first attempt with this dynamic programming problem. You're in great form today!

Approach

Current: Dynamic Programming

Suggested: Dynamic Programming

Key Idea: Using DP to track minimum swaps where each state depends on whether the current index was swapped or not.

Runtime

81 ms | Beats 48.80%

Memory

124.12 MB | Beats 54.59%

287. Find the Duplicate Number

Code

C v Auto

```
1 int findDuplicate(int* nums, int numsSize) {
2     int slow = nums[0];
3     int fast = nums[0];
4     do {
5         slow = nums[slow];
6         fast = nums[nums[fast]];
7     } while (slow != fast);
8     slow = nums[0];
9
10    while (slow != fast)
11    {
12        slow = nums[slow];
13        fast = nums[fast];
14    }
15    return slow;
16 }
```

Accepted X

Accepted 59 / 59 testcases passed

Avinash_Acharya1 submitted at May 30, 2026 19:36

Analysis Solution

Runtime

8 ms | Beats 21.13%

Memory

15.02 MB | Beats 15.76%